MECHATRONICS AND IOT

ASSIGNMENT REPORT - 8

TITLE:

Design and develop an IoT-Based HVAC Temperature and Humidity Monitoring System. The system continuously monitors environmental conditions to improve HVAC performance and energy management

DONE BY ,

HARI KRISHNA JAGADEESH-24MER011

HARISH DHARMALINGAM – 24MER014

RANJITH KUMAR P – 24MER048

RISHI KESAV A – 24MER049

VENKATRAMANAN B – 24MER061

Project Overview:

The Smart Refrigerator Monitoring System is an IoT-based system designed to continuously monitor the temperature and humidity inside a refrigerator to maintain suitable conditions for food preservation. The system uses a DHT11 sensor to measure the internal temperature and humidity, while an ESP8266 NodeMCU processes the sensor data.

The measured values are displayed on an OLED display, allowing the user to monitor refrigerator conditions in real time. The system compares the measured parameters with predefined threshold values and identifies normal or abnormal conditions. A red LED and buzzer can provide an alert when the temperature or humidity exceeds the desired range.

The ESP8266 can also use its built-in Wi-Fi connectivity to transmit the collected data to an IoT/cloud platform for remote monitoring and analysis. The proposed system provides a low-cost, real-time and automated solution for improving food preservation, reducing spoilage and supporting efficient refrigerator operation.

Problem Statement:

Maintaining suitable temperature and humidity conditions inside a refrigerator is essential for preserving food quality and preventing spoilage. Conventional refrigerators may not provide continuous monitoring or immediate alerts when the internal conditions become unsuitable.

Temperature fluctuations, excessive humidity, or prolonged exposure to inappropriate conditions can reduce food shelf life and increase food wastage. Manual monitoring is also inconvenient and may result in delayed detection of abnormal conditions.

Therefore, there is a need for a low-cost, real-time monitoring system that can continuously measure temperature and humidity, identify abnormal conditions and provide timely alerts to improve food preservation, reduce spoilage and support energy-efficient refrigerator operation.

Proposed Solution:

The proposed system uses a DHT11 sensor and ESP8266 NodeMCU to continuously monitor the temperature and humidity inside the refrigerator. The DHT11 sensor

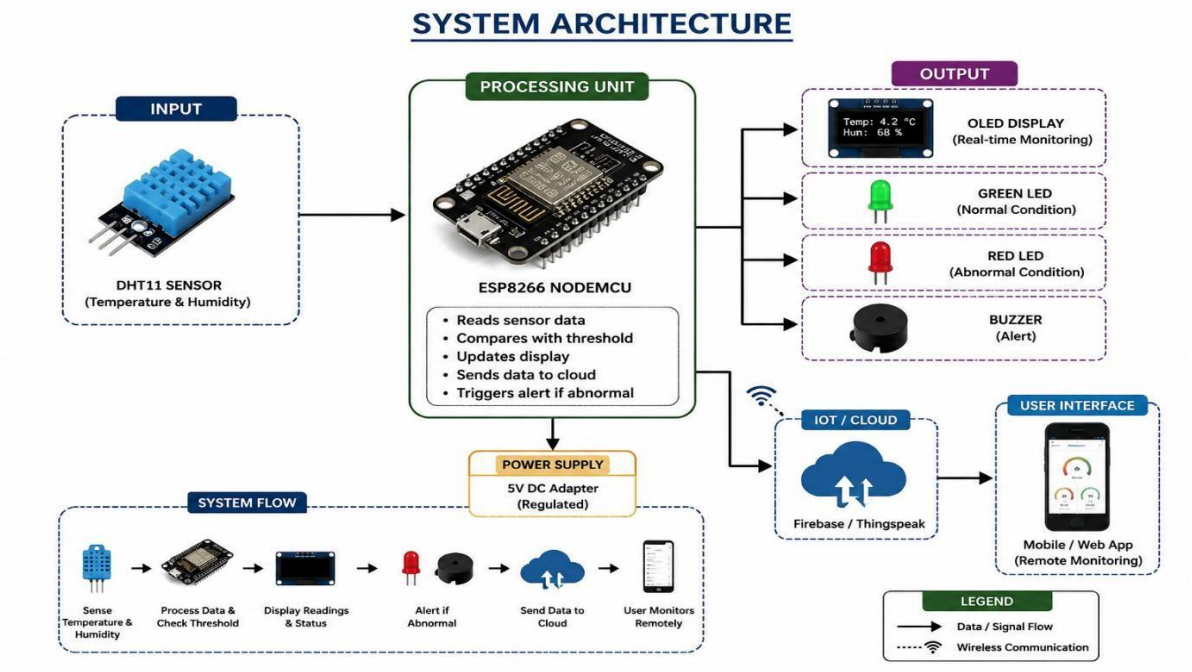
measures the internal environmental conditions and sends the readings to the ESP8266 for processing.

The ESP8266 compares the measured temperature and humidity with predefined threshold values to determine whether the refrigerator is operating under Normal or Abnormal conditions. The current readings and system status are displayed on an OLED display for real-time monitoring.

When the temperature or humidity exceeds the predefined limits, a red LED and buzzer are activated to alert the user. Under normal conditions, a green LED indicates that the refrigerator environment is within the desired range.

The ESP8266 can also use its built-in Wi-Fi connectivity to transmit the collected data to an IoT/cloud platform for remote monitoring and analysis. The proposed solution provides a low-cost, real-time and automated monitoring system that helps improve food preservation, reduce spoilage and support efficient refrigerator operation.

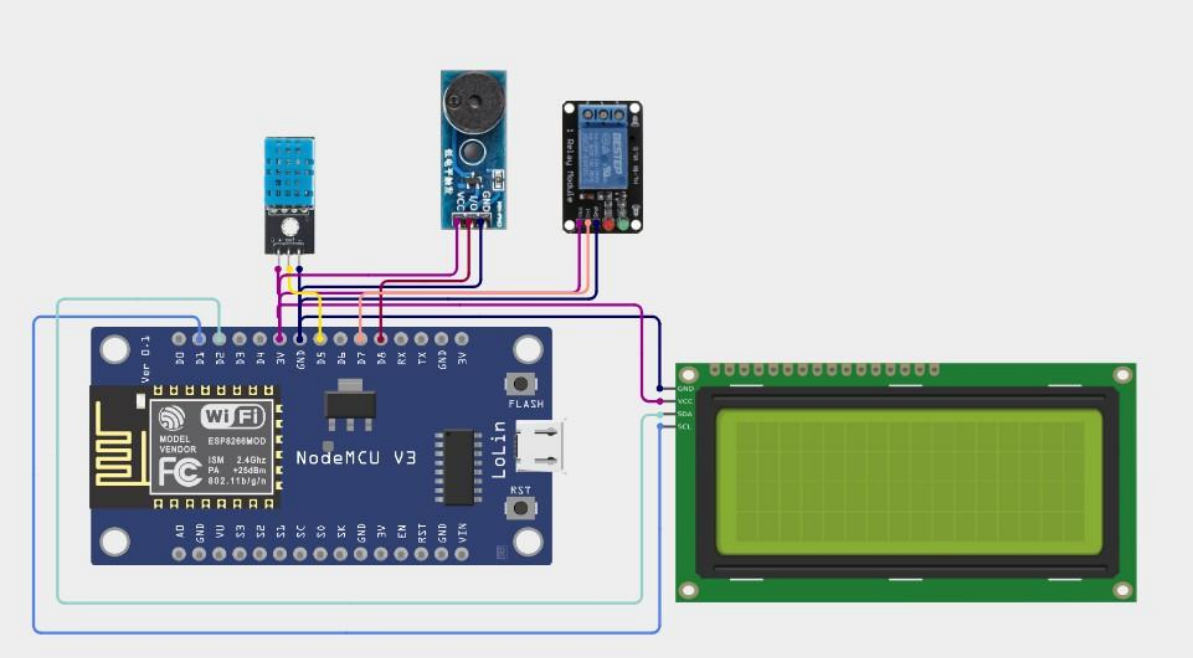
System Architecture:



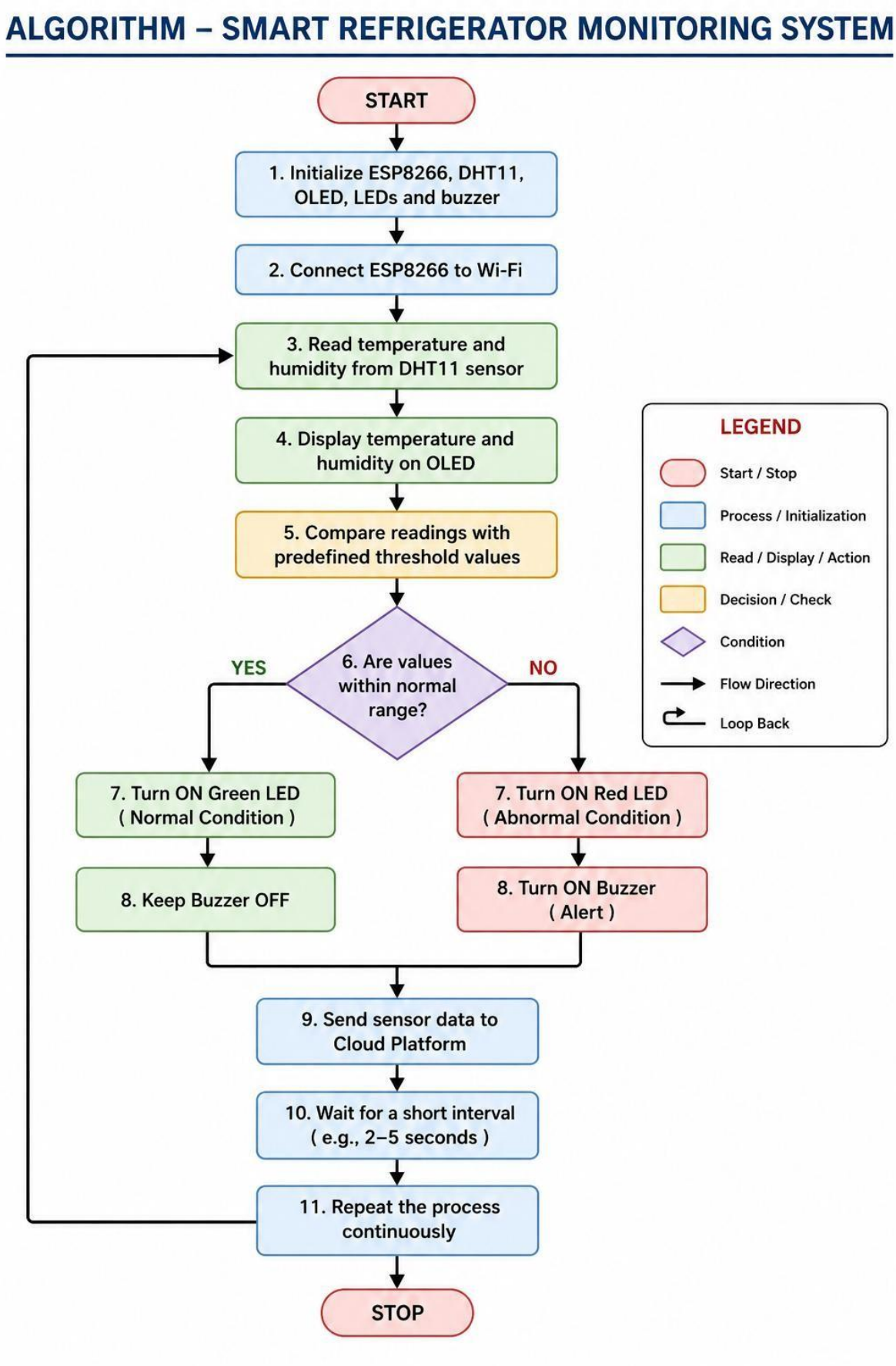
Circuit Table :

Component	Pin/Terminal	ESP8266 Connection	Purpose
DHT11 Sensor	VCC	3.3V	Power supply
LCD Display	VCC	3.3V	Safe indication
ESP8266	VIN/USB	5V USB	Hazardous indication
Buzzer	Positive (+)	GPIO 14	Alert

Circuit Design:



Algorithm:


ALGORITHM – SMART REFRIGERATOR MONITORING SYSTEM
START
1. Initialize ESP8266, DHT11, OLED, LEDs and buzzer
2. Connect ESP8266 to Wi-Fi
3. Read temperature and humidity from DHT11 sensor
4. Display temperature and humidity on OLED
5. Compare readings with predefined threshold values
6. Are values within normal range?
YES
NO
7. Turn ON Green LED (Normal Condition)
8. Keep Buzzer OFF
7. Turn ON Red LED (Abnormal Condition)
8. Turn ON Buzzer (Alert)
9. Send sensor data to Cloud Platform
10. Wait for a short interval (e.g., 2–5 seconds)
11. Repeat the process continuously
STOP
LEGEND
Start / Stop
Process / Initialization
Read / Display / Action
Decision / Check
Condition
Flow Direction
Loop Back

Coding:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <DHT.h>
#include <ESP8266WiFi.h>
#include "Adafruit_MQTT.h"
#include "Adafruit_MQTT_Client.h"
// ============================================================
// WIFI SETTINGS
// ============================================================
#define WIFI_SSID       "Hari"
#define WIFI_PASSWORD   "11111111"
// ============================================================
// ADAFRUIT IO SETTINGS
// ============================================================
#define AIO_USERNAME    "Hari_87789"
#define AIO_KEY         "aio_mTwj83UKedBOZF7ynQuOfANn9JK0"
#define AIO_SERVER      "io.adafruit.com"
#define AIO_SERVERPORT  1883
// ============================================================
// DHT11 SETTINGS
// ============================================================
#define DHT_PIN         14       // D5 / GPIO14
#define DHT_TYPE        DHT11
DHT dht(DHT_PIN, DHT_TYPE);
// ============================================================
// RELAY
// Active LOW
// LOW  = ON
```

```
// HIGH = OFF
// ============================================================
#define RELAY_PIN       12       // D6 / GPIO12
// ============================================================
// BUZZER
// Active LOW
// LOW  = ON
// HIGH = OFF
// ============================================================
#define BUZZER_PIN      13       // D7 / GPIO13
// ============================================================
// LCD
// ============================================================
// Common I2C address = 0x27
LiquidCrystal_I2C lcd(0x27, 16, 2);
// ============================================================
// MQTT
// ============================================================
WiFiClient client;
Adafruit_MQTT_Client mqtt(
&client,
  AIO_SERVER,
  AIO_SERVERPORT,
  AIO_USERNAME,
  AIO_KEY
);
// ============================================================
// ADAFRUIT IO FEEDS
// ============================================================
Adafruit_MQTT_Publish temperatureFeed =
```

```
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/refrigerator-temperature"
  );
Adafruit_MQTT_Publish humidityFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/refrigerator-humidity"
  );
Adafruit_MQTT_Publish statusFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/refrigerator-status"
  );
// ========================================================
// TEMPERATURE SETTINGS
// ========================================================
// Cooling turns ON when temperature goes above this
#define HIGH_TEMP 8.0
// Cooling turns OFF when temperature comes down to this
#define LOW_TEMP 5.0
// ========================================================
// VARIABLES
// ========================================================
bool coolingON = false;
bool highTemperatureAlert = false;
// ========================================================
// WIFI CONNECTION
// ========================================================
void connectWiFi()
```

```
{
  Serial.println();
  Serial.println("Connecting to WiFi...");
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED)
  {
    delay(500);
    Serial.print(".");
  }
  Serial.println();
  Serial.println("WiFi Connected!");
  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());
}
// ====================================================
// ADAFRUIT IO CONNECTION
// ====================================================
void connectMQTT()
{
  if (mqtt.connected())
  {
    return;
  }
  Serial.println("Connecting to Adafruit IO...");
  int8_t ret;
  while ((ret = mqtt.connect()) != 0)
  {
    Serial.println(mqtt.connectErrorString(ret));
    mqtt.disconnect();
    delay(5000);
```

```
    Serial.println("Retrying...");
  }
  Serial.println("Adafruit IO Connected!");
}
// ============================================================
// RELAY ON
// ============================================================
void relayON()
{
  digitalWrite(RELAY_PIN, LOW);
  coolingON = true;
  Serial.println("COOLING: ON");
}
// ============================================================
// RELAY OFF
// ============================================================
void relayOFF()
{
  digitalWrite(RELAY_PIN, HIGH);
  coolingON = false;
  Serial.println("COOLING: OFF");
}
// ============================================================
// BUZZER ON
// ============================================================
void buzzerON()
{
  digitalWrite(BUZZER_PIN,  LOW);
}
// ============================================================
```

```
// BUZZER OFF
// ========================================================
void buzzerOFF()
{
  digitalWrite(BUZZER_PIN, HIGH);
}
// ========================================================
// UPDATE LCD
// ========================================================
void displayNormal(float temperature, float humidity)
{
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Temp: ");
  lcd.print(temperature, 1);
  lcd.print((char)223);
  lcd.print("C");
  lcd.setCursor(0, 1);
  lcd.print("Hum: ");
  lcd.print(humidity, 1);
  lcd.print("%");
}
// ========================================================
// DISPLAY HIGH TEMPERATURE
// ========================================================
void displayHighTemperature(float temperature)
{
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("HIGH TEMP!");
```

```
  lcd.setCursor(0, 1);
  lcd.print("Temp: ");
  lcd.print(temperature, 1);
  lcd.print((char)223);
  lcd.print("C");
}
// ========================================================
// SEND DATA TO ADAFRUIT IO
// ========================================================
void sendToCloud(float temperature, float humidity)
{
  Serial.println("Sending data to Adafruit IO...");
  if (temperatureFeed.publish(temperature))
  {
    Serial.println("Temperature  sent");
  }
  else
  {
    Serial.println("Temperature upload failed");
  }
  if (humidityFeed.publish(humidity))
  {
    Serial.println("Humidity sent");
  }
  else
  {
    Serial.println("Humidity upload failed");
  }
  if (coolingON)
  {
```

```
    statusFeed.publish("COOLING ON");
  }
  else if (highTemperatureAlert)
  {
    statusFeed.publish("HIGH  TEMPERATURE");
  }
  else
  {
    statusFeed.publish("NORMAL");
  }
}
// ================================================
// SETUP
// ================================================
void setup()
{
  Serial.begin(115200);
  delay(1000);
  Serial.println();
  Serial.println("========================================");
  Serial.println(" SMART REFRIGERATOR MONITORING SYSTEM");
  Serial.println("========================================");
  // ================================================
  // PIN MODES
  // ================================================
  pinMode(RELAY_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  // ================================================
  // INITIAL RELAY
  // Active LOW
```

```
// HIGH = OFF
// ================================================
relayOFF();
// ================================================
// INITIAL BUZZER
// Active LOW
// HIGH = OFF
// ================================================
buzzerOFF();
// ================================================
// DHT11
// ================================================
dht.begin();
// ================================================
// LCD
// ================================================
Wire.begin(4, 5);      // SDA = D2, SCL = D1
lcd.init();
lcd.backlight();
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("SMART FRIDGE");
lcd.setCursor(0, 1);
lcd.print("Starting...");
delay(2000);
// ================================================
// WIFI
// ================================================
connectWiFi();
// ================================================
```

```
  // ADAFRUIT IO
  // ================================================
  connectMQTT();
  // ================================================
  // READY
  // ================================================
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("System Ready");
  lcd.setCursor(0, 1);
  lcd.print("Monitoring...");
  Serial.println();
  Serial.println("------------------------");
  Serial.println("SYSTEM READY");
  Serial.println("Cooling: OFF");
  Serial.println("------------------------");
  delay(2000);
}
// ================================================
// MAIN LOOP
// ================================================
void loop()
{
  // ================================================
  // WIFI CHECK
  // ================================================
  if (WiFi.status() != WL_CONNECTED)
  {
    connectWiFi();
  }
```

```
// ======================================================
// ADAFRUIT IO CHECK
// ======================================================
if (!mqtt.connected())
{
  connectMQTT();
}
mqtt.processPackets(10);
mqtt.ping();
// ======================================================
// READ DHT11
// ======================================================
float temperature = dht.readTemperature();
float humidity = dht.readHumidity();
// ======================================================
// CHECK SENSOR
// ======================================================
if (isnan(temperature) || isnan(humidity))
{
  Serial.println("DHT11 ERROR!");
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("DHT11 ERROR");
  lcd.setCursor(0, 1);
  lcd.print("Check Sensor");
  delay(2000);
  return;
}
// ======================================================
// SERIAL MONITOR
```

```
// ================================================
Serial.println();
Serial.println("------------------------");
Serial.print("Temperature: ");
Serial.print(temperature);
Serial.println(" C");
Serial.print("Humidity: ");
Serial.print(humidity);
Serial.println(" %");
// ================================================
// TEMPERATURE CONTROL
// ================================================
// ------------------------------
// TEMPERATURE TOO HIGH
// ------------------------------
if (temperature > HIGH_TEMP)
{
  highTemperatureAlert = true;
  Serial.println("STATUS: HIGH TEMPERATURE");
  // Turn cooling ON
  relayON();
  // Buzzer ON
  buzzerON();
  // LCD warning
  displayHighTemperature(temperature);
}
// ------------------------------
// TEMPERATURE NORMAL
// ------------------------------
else if (temperature <= LOW_TEMP)
```

```
{
  highTemperatureAlert = false;
  Serial.println("STATUS: NORMAL");
  // Turn cooling OFF
  relayOFF();
  // Buzzer OFF
  buzzerOFF();
  // Normal LCD
  displayNormal(temperature, humidity);
}
// ------------------------------
// BETWEEN 5°C AND 8°C
// ------------------------------
else
{
  Serial.println("STATUS: COOLING");
  // Keep current relay state
  if (coolingON)
  {
    relayON();
  }
  // Buzzer OFF
  buzzerOFF();
  // Display
  displayNormal(temperature, humidity);
}
// ================================================
// SEND DATA TO ADAFRUIT IO
// ================================================
sendToCloud(temperature, humidity);
```

```
  // ================================================
  // WAIT
  // ================================================
  delay(5000);
}
```

System Working:

- The Smart Refrigerator Monitoring System uses a DHT11 sensor to continuously measure the temperature and humidity inside the refrigerator. The sensor sends the collected data to the ESP8266 NodeMCU, which processes and analyzes the readings.
- The measured temperature and humidity are displayed on the OLED display for real-time monitoring. The ESP8266 compares the readings with predefined threshold values to determine whether the refrigerator is operating under normal conditions.
- When the temperature and humidity are within the desired range, the green LED remains ON and the buzzer remains OFF. If either parameter exceeds the predefined limit, the red LED and buzzer are activated to alert the user about abnormal conditions.
- The ESP8266 can also transmit the collected data through Wi-Fi to an IoT/cloud platform for remote monitoring and analysis. The system continuously repeats the sensing, processing and alerting process, helping maintain suitable conditions for food preservation and efficient refrigerator operation.

Bill Of Material:

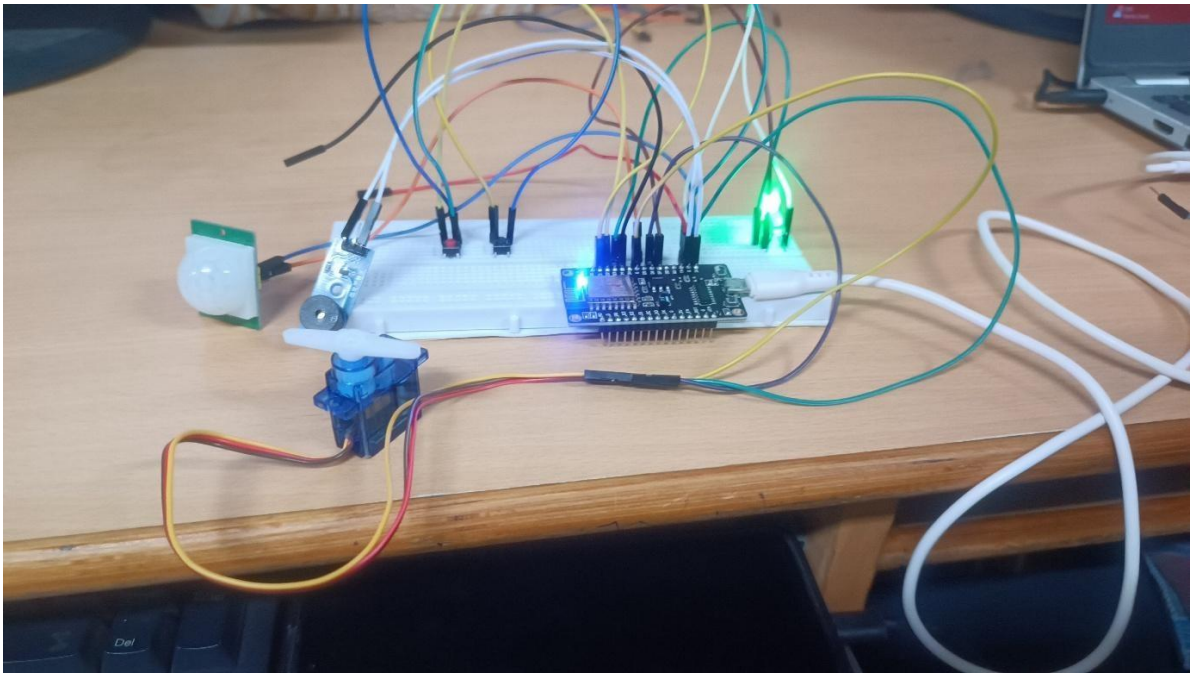
S. no	Component	Specification	Quantity	Cost
1	ESP8266	Node MCU ESP8266	1	Rs. 450
2	DHT11 Sensor	Temperature & Humidity Sensor	1	Rs. 100
3	Servo Motor	SG90	1	Rs. 100

4	Bread Board	Standard 830-point	1	Rs. 100
5	Jumper Wire	Male-Male / Male-Female	1 set	₹50
		Total		₹800

Prototype Implementation:

- The prototype was implemented using an ESP8266 NodeMCU, DHT11 temperature and humidity sensor, OLED display, green LED, red LED and buzzer. The DHT11 sensor was placed inside the refrigerator model to continuously measure the internal temperature and humidity.
- The sensor readings are collected and processed by the ESP8266. The temperature and humidity values are displayed on the OLED screen for real-time monitoring. The ESP8266 compares the readings with predefined threshold values to determine the refrigerator's operating condition.
- Under normal conditions, the green LED remains ON and the buzzer remains OFF. If the temperature or humidity exceeds the specified limits, the red LED and buzzer are activated to provide an immediate warning.
- The ESP8266 can also use its built-in Wi-Fi connectivity to transmit the sensor data to an IoT/cloud platform for remote monitoring. The prototype continuously updates the readings and provides alerts whenever abnormal conditions are detected.

Prototype flow:
DHT11 → ESP8266 → Data Processing → OLED Display + LED/Buzzer Alert → Wi-Fi → Cloud Monitoring



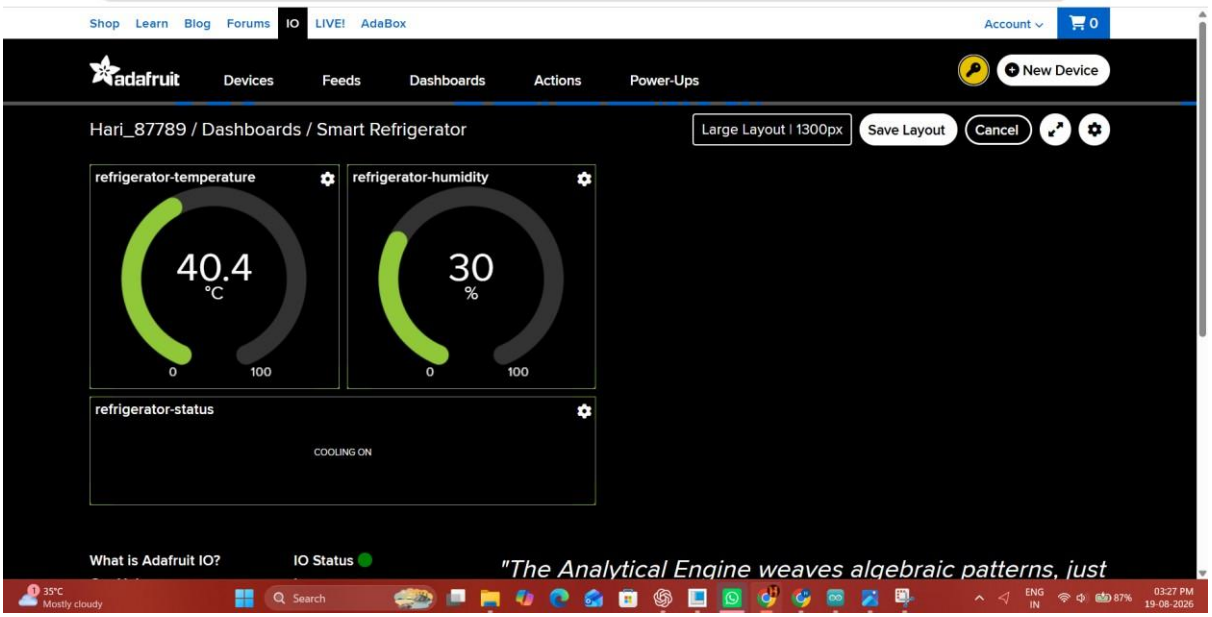
Results and Performance:

- The developed Smart Refrigerator Monitoring System successfully monitored the internal temperature and humidity using the DHT11 sensor and ESP8266 NodeMCU. The sensor readings were processed in real time and displayed on the OLED display, allowing continuous monitoring of refrigerator conditions.
- The system successfully identified normal and abnormal conditions based on predefined temperature and humidity limits. During normal conditions, the green LED indicated proper operating conditions, while the red LED and buzzer were activated when the measured values exceeded the specified limits.

Performance Analysis

Parameter	Result
Temperature Monitoring	Successfully measured
Humidity Monitoring	Successfully measured
OLED Display	Real-time readings
Normal Condition	Green LED ON
Abnormal Condition	Red LED + Buzzer ON
Response	Real-time
Data Monitoring	Continuous
Wi-Fi Connectivity	Supported

Overall, the prototype demonstrated a **low-cost, reliable and real-time monitoring system** for refrigerator conditions. It can help detect abnormal temperature and humidity conditions early, thereby improving **food preservation, reducing spoilage and supporting efficient refrigerator operation**.


Shop Learn Blog Forums IO LIVE! AdaBox
Account 0
adafruit
Devices Feeds Dashboards Actions Power-Ups
New Device
Hari_87789 / Dashboards / Smart Refrigerator
Large Layout | 1300px
Save Layout
Cancel
refrigerator-temperature
40.4
°C
0
100
refrigerator-humidity
30
%
0
100
refrigerator-status
COOLING ON
What is Adafruit IO?
IO Status
"The Analytical Engine weaves algebraic patterns, just
35°C
Mostly cloudy
Search
ENG
IN
87%
03:27 PM
19-08-2026